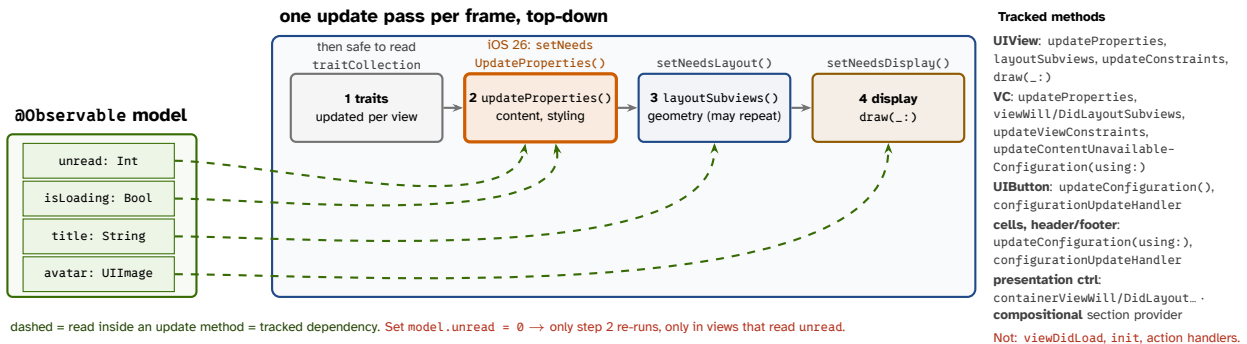


In one line: UIKit **subscribes for you:** an `@Observable` property (or a trait) you **read inside an update method** becomes a dependency; when it changes, UIKit invalidates **only that view** and re-runs **only that method** in the next update pass — no `setNeeds...`... On by default in **iOS 26**; iOS 18 opts in with `UIObservationTrackingEnabled`.



How it works

- **Observation tracking:** each update method runs inside Observation's access tracking; every property `get` registers the view, a later `set` schedules that view's matching invalidation.
- **iOS 26:** always on. **iOS 18:** `Info.plist` `UIObservationTrackingEnabled = YES` tracks the *existing* methods only.
- **`updateProperties()`** (iOS 26, `UIView` + `UIViewController`): content, styling, behaviour. Runs after traits, just before `layoutSubviews`, but is invalidated **independently** — a text change no longer forces a layout. It may invalidate layout; layout runs right after.
- **`.flushUpdates`** (iOS 26): `UIView.animate(options: .flushUpdates) { ... }` applies pending updates before and after the block — change a constraint or a model property inside; no `layoutIfNeeded()`.
- **Traits** — iOS 17: `registerForTraitChanges([UITrait...], handler:)` replaces the deprecated `traitCollectionDidChange`; removed automatically at dealloc. Custom trait = `UITraitDefinition` (`defaultValue`) + accessors on `UITraitCollection` and `UIMutableTraits`; set it with `traitOverrides`, it flows *down*; `UITraitBridgedEnvironmentKey` mirrors it into SwiftUI. iOS 18: **automatic trait tracking** in the same update methods.
- **`UIButton.Configuration`** (iOS 15): `.plain/.gray/.tinted/.filled()`... + `subtitle`, `imagePlacement`, `showsActivityIndicator`, `cornerStyle`; per-state tweaks in `configurationUpdateHandler`. iOS 26: `.glass()`, `.prominentGlass()`, `.clearGlass()`, `.prominentClearGlass()`.
- **`UIContentUnavailableConfiguration`** (iOS 17): `.empty()/loading()/search()` + `image`, `text`, `secondaryText`, `button`; assign `vc.contentUnavailableConfiguration` (`nil` hides) inside `updateContentUnavailableConfiguration(using:)`; before 26 trigger it with `setNeedsUpdateContentUnavailableConfiguration()`.

Example — one model drives badge, spinner, empty state

```

@Observable final class Inbox { var unread = 0; var loading = true }
final class InboxVC: UIViewController {
    let inbox = Inbox(); let refresh = UIButton(configuration: .filled())
    override func updateProperties() { // iOS 26, tracked
        super.updateProperties()
        tabBarItem.badgeValue = inbox.unread > 0 ? "\(inbox.unread)" : nil
        refresh.configuration?.showsActivityIndicator = inbox.loading
    }
    override func updateContentUnavailableConfiguration(
        using state: UIContentUnavailableConfigurationState) {
        contentUnavailableConfiguration = inbox.loading ? .loading() : nil
    }
    func markAllRead() { // animated re-run
        UIView.animate(options: .flushUpdates) { self.inbox.unread = 0 }
    }
    func observeWidth() { // iOS 17: once, viewDidLoad
        registerForTraitChanges([UITraitHorizontalSizeClass.self]) {
            (self: Self, previous: UITraitCollection) in self.layout()
        }
    }
}
  
```

What landed when — say the version

15	<code>UIButton.Configuration</code> · cell configurationUpdateHandler · keyboardLayoutGuide · pop-up buttons
16	<code>UIHostingConfiguration</code> · <code>UIEditMenuInteraction</code> · multi-item context menus
17	<code>viewIsAppearing</code> (back to 13) · trait system · <code>UIContentUnavailableConfiguration</code> · <code>#Preview</code> · <code>addSymbolEffect(.bounce)</code> · <code>uniformAcrossSiblings</code> · palette menus
18	<code>auto trait</code> tracking · observation opt-in · <code>UIUpdateLink</code> · <code>UITab/UITabGroup</code> · <code>UIBackgroundConfiguration</code> · <code>listCell()</code> · <code>UIView.animate(.spring)</code> (SwiftUI Animation)
26	observation default · <code>updateProperties</code> · <code>.flushUpdates</code> · typed <code>NotificationCenter.Message</code> · <code>UIMainMenuSystem</code> · glass buttons · <code>UIHostingSceneDelegate</code> · split-view inspector

Next: the release after iOS 26 refuses to launch a UIKit app built with the latest SDK that has not adopted the **UIScene life cycle** (WWDC25).

Traps

- ! Model read in `viewDidLoad` or an action is **not** tracked — the label never updates. Read it in an update method.
- ! Only an `@Observable` **class** read *through* the handler is tracked; a handler that captured a *struct copy* of the item stays frozen.
- ! iOS 18 target: the `plist` key covers layout/draw/config handlers, but `updateProperties` and `.flushUpdates` need `if #available(iOS 26, *)`.
- ! Constraint change in `UIView.animate` with neither `layoutIfNeeded()` nor `.flushUpdates` → it jumps.
- ! `registerForTraitChanges` inside `layoutSubviews` re-registers every pass — once.
- ! `var c = button.configuration; c?.title = "Go"` edits a **copy** — assign it back.
- ! Mixing `setTitle(_:for:)` / edge insets with a configuration: the configuration wins — style a button one way only.

Remember

“Read it in an update method and you are subscribed.”

Order: **traits** → **properties** → **layout** → **draw**, each with its own `setNeeds...`